

Phase 1: Making an agent

Requirements:

Python version 3.10++

Tensorflow

CUDA

The GPU **MUST** be detectable by python otherwise this whole project will not be possible on that machine.

Constraints:

Most of the constraints mentioned here would be relating to hardware, specifically the RAM of the GPU on said machine otherwise referred to as the VRAM. The first section of this document will be addressing this when selecting a model/models for use in this project. The purpose of the agent should also be considered when selecting model/models. Please keep this in mind. While WSL is not a requirement, the document in question is using examples of environments running on WSL.

If you are using VRAM less than 6GB, please continue reading otherwise you may skip to page 3 for some model recommendations as well as some performance metrics. Actual installation guide starts at page 10.

Phase 1: Making an agent

Understanding 4GB VRAM Constraints:

VRAM (Video RAM) is the memory on your GPU. For local LLMs, it is the single most important hardware constraint — more important than CPU speed or system RAM. The model must fit entirely in VRAM to run at a usable speed.

Key Point: VRAM is a hard limit. If the model does not fit, Ollama will either refuse to load it or fall back to CPU inference, which is 5–20x slower.

With 4GB VRAM, you are working in the entry-level tier. This limits you to models in the 1B–4B parameter range at Q4_K_M quantization. Here is what that means in practice:

VRAM Budget	Usable VRAM for Model	Why Not Full 4GB?
4GB GPU (e.g. RTX 3050 4GB)	~3.0–3.4 GB	~0.6–1.0 GB reserved for OS, driver, and Ollama runtime overhead
4GB GPU with Windows running	~2.8–3.2 GB	Windows desktop GPU usage reduces headroom further
4GB GPU on WSL2	~3.0–3.4 GB	WSL2 has minimal GPU overhead beyond the driver baseline

Warning: Never attempt to run a 7B model at Q4_K_M on a 4GB GPU. A Q4_K_M 7B model requires approximately 4.5–5.0 GB of VRAM — it will either fail to load or run so slowly on CPU fallback that it is unusable. Always leave headroom for your GPU to breath otherwise it w

Note: Llama 3.1 8B at Q2_K (3.2 GB) can technically load on some 4GB cards, but the quality loss at Q2 quantization is significant enough that a smaller, better-quantized model will produce superior results. Always prefer a smaller model at Q4_K_M over a larger model at Q2_K.

Phase 1: Making an agent

Recommended Models by Task Category:

Note: You may opt to use bigger models IF VRAM allows it. Or different models if VRAM allows it. You are not limited by these recommendations.

General Chat and Reasoning:

Qwen3 4B

Pull: `ollama pull qwen3:4b` **VRAM:** ~2.8 GB **Est. TPS:** 30–40 TPS

Best for: General chat, reasoning, instruction following, tool use

Why: The strongest general-purpose model at the 4B tier as of 2026. Supports both thinking (chain-of-thought) and non-thinking modes. Benchmark scores rival models twice its size.

Gemma 4 E4B

Pull: `ollama pull gemma4:e4b` **VRAM:** ~2.6 GB **Est. TPS:** 35–45 TPS

Best for: Instruction following, conversational tasks, agentic workflows

Why: Google's latest efficient architecture. Strong reasoning for its size, native function calling support, and 128K context window. Good alternative to Qwen3 4B.

Llama 3.2 3B

Pull: `ollama pull llama3.2:3b` **VRAM:** ~2.0 GB **Est. TPS:** 40–55 TPS

Best for: Quick tasks, low-latency chat, simple Q&A

Why: Meta's compact model. Smaller and faster than 4B options. Good for tasks where speed matters more than depth. 128K context window.

Coding:

For code generation, debugging, scaffolding, boilerplate, and code explanation. Specialized coding models significantly outperform general models of the same size on code tasks

Qwen2.5-Coder 3B

Pull: `ollama pull qwen2.5-coder:3b` **VRAM:** ~2.0 GB **Est. TPS:** 40–55 TPS

Best for: Code generation, debugging, scaffolding — Python, C#, JavaScript, and 40+ languages

Why: The top coding model at the 3B tier. Trained specifically on code. Significantly outperforms general 3B models on coding benchmarks. Handles C#, Python, WPF, and most mainstream languages well.

Qwen3-Coder 3B

Pull: `ollama pull qwen3-coder` **VRAM:** ~2.2 GB **Est. TPS:** 35–45 TPS

Phase 1: Making an agent

Best for: Advanced code reasoning, multi-step debugging, code explanation

Why: *Newer than Qwen2.5-Coder. Better at reasoning through complex code logic. If available in Ollama registry, prefer this over Qwen2.5-Coder 3B for debugging tasks.*

Phi-4 Mini

Pull: `ollama pull phi4-mini` **VRAM:** ~2.5 GB **Est. TPS:** 35–45 TPS

Best for: Code completion, general coding assistance, math-heavy code

Why: *Microsoft's compact model. Punches above its weight on reasoning and code tasks. Good fallback if Qwen models are unavailable or underperform on specific tasks.*

Summarization:

For condensing documents, lecture notes, articles, and code comments into shorter, structured summaries.

Qwen3 4B

Pull: `ollama pull qwen3:4b` **VRAM:** ~2.8 GB **Est. TPS:** 30–40 TPS

Best for: Document summarization, note condensing, structured output

Why: *The same model recommended for general chat. Its strong instruction following makes it reliable for summarization tasks with specific format requirements (bullet points, headers, etc.).*

Llama 3.2 3B

Pull: `ollama pull llama3.2:3b` **VRAM:** ~2.0 GB **Est. TPS:** 40–55 TPS

Best for: Quick summarization, shorter documents

Why: *Faster than Qwen3 4B. Good for summarizing shorter content where speed is preferred over depth. Less reliable on very long documents.*

Warning: *All 4GB-viable models have limited context windows compared to 7B+ models. Qwen3 4B and Llama 3.2 3B support 128K context, but VRAM usage increases with context length. For very long documents (10,000+ tokens), reduce the context window via Ollama settings or split the document into chunks.*

Phase 1: Making an agent

RAG and Embedding

Retrieval-Augmented Generation (RAG) requires two components: an embedding model (converts text to vectors for search) and a generation model (produces answers from retrieved context). Embedding models are very small and have minimal VRAM requirements.

nomic-embed-text

Pull: `ollama pull nomic-embed-text` **VRAM:** ~0.3 GB **Est. TPS:** N/A (embedding only)

Best for: Generating text embeddings for RAG pipelines, semantic search, document indexing

Why: *The standard embedding model for Ollama-based RAG setups. Extremely small VRAM footprint. Used alongside a generation model — both can coexist in 4GB VRAM since the embedding model is near zero.*

mxbai-embed-large

Pull: `ollama pull mxbai-embed-large` **VRAM:** ~0.5 GB **Est. TPS:** N/A (embedding only)

Best for: Higher quality embeddings for RAG, better semantic retrieval accuracy

Why: *Slightly larger than nomic-embed-text but produces better quality embeddings. Still small enough to coexist with a 3B generation model within 4GB total.*

Note: *For a complete RAG setup: pull nomic-embed-text for indexing, then load Qwen3 4B or Llama 3.2 3B for generation. Do not load both a large generation model and an embedding model simultaneously.*

Math and Logic Reasoning

For step-by-step problem solving, mathematical reasoning, and logical deduction tasks.

Qwen3 4B (Thinking Mode)

Pull: `ollama pull qwen3:4b` **VRAM:** ~2.8 GB **Est. TPS:** 20–30 TPS (thinking is slower)

Best for: Math problems, logic puzzles, step-by-step reasoning

Why: *Qwen3 4B includes a built-in thinking mode that enables explicit chain-of-thought reasoning. Activate it by prompting: 'Think step by step.' TPS drops during thinking but answer quality improves significantly for math and logic tasks.*

DeepSeek-R1 1.5B

Pull: `ollama pull deepseek-r1:1.5b` **VRAM:** ~1.1 GB **Est. TPS:** 50–70 TPS

Best for: Math reasoning, logic tasks — fast and lightweight

Why: *A distilled reasoning model from DeepSeek. Extremely small VRAM footprint. Reasoning quality for math tasks exceeds what the parameter count suggests. Good option when VRAM is very tight.*

Phase 1: Making an agent

Creative Writing

For drafting, storytelling, content writing, and structured text generation.

Qwen3 4B

Pull: `ollama pull qwen3:4b` **VRAM:** ~2.8 GB **Est. TPS:** 30–40 TPS

Best for: Lesson content drafting, structured writing, email drafting

Why: *The same all-purpose recommendation. At the 4B tier, Qwen3 4B produces the most coherent and stylistically consistent creative output. Suitable for drafting explanations, and short-form content.*

Gemma 4 E2B

Pull: `ollama pull gemma4:e2b` **VRAM:** ~1.5 GB **Est. TPS:** 50–65 TPS

Best for: Quick drafts, short-form writing, fast iteration

Why: *Google's smallest Gemma 4 variant. Very fast. Good for quick drafts where iteration speed matters more than depth. Not suitable for long-form or highly structured content.*

Phase 1: Making an agent

VRAM Requirements and Performance Summary

All figures are for Q4_K_M quantization at default Ollama settings. TPS estimates are for a 4GB mobile GPU (RTX 3050 4GB / RTX 2050 4GB class). Desktop GPUs of the same VRAM will be 10–20% faster due to higher memory bandwidth.

Model	Ollama Tag	VRAM (Q4_K_M)	Est. TPS	Context	Best Use
Qwen3 4B	qwen3:4b	~2.8 GB	30–40	128K	General, reasoning, summarization, creative
Gemma 4 E4B	gemma4:e4b	~2.6 GB	35–45	128K	General chat, agentic tasks
Phi-4 Mini	phi4-mini	~2.5 GB	35–45	128K	Coding, math, reasoning
Llama 3.2 3B	llama3.2:3b	~2.0 GB	40–55	128K	Fast chat, summarization
Qwen2.5-Coder 3B	qwen2.5-coder:3b	~2.0 GB	40–55	32K	Coding — primary recommendation
Qwen3-Coder 3B	qwen3-coder	~2.2 GB	35–45	128K	Coding — advanced reasoning
DeepSeek-R1 1.5B	deepseek-r1:1.5b	~1.1 GB	50–70	128K	Math and logic reasoning
Gemma 4 E2B	gemma4:e2b	~1.5 GB	50–65	128K	Fast drafts, quick tasks
nomic-embed-text	nomic-embed-text	~0.3 GB	N/A	8K	RAG embeddings only
mxbai-embed-large	mxbai-embed-large	~0.5 GB	N/A	512	RAG embeddings — higher quality

Note: TPS figures are estimates based on 4GB mobile GPU class hardware. Actual performance varies by system RAM, CPU, driver version, and background processes. Use these as planning figures, not guarantees.

Phase 1: Making an agent

No Simultaneous Model Loading

On 4GB VRAM, only one model can be loaded at a time. Loading a second model while the first is active will cause the first to be evicted from VRAM. Plan your workflow accordingly — use Ollama's OLLAMA_KEEP_ALIVE=0 setting to ensure models release VRAM promptly after each session.

CPU Fallback Is Not a Solution

If a model does not fit in VRAM, Ollama falls back to CPU inference. On typical student laptops, CPU inference runs at 1–3 TPS — too slow for interactive use. If a model requires CPU fallback, it should be replaced with a smaller model that fits entirely in VRAM.

Download Sources and Quantization Notes

How to Pull Models via Ollama

All models in this guide are available directly through Ollama. Install Ollama first, then pull any model with a single command:

```
ollama pull <model_tag>
```

To pull a specific quantization variant, append the quantization tag:

```
ollama pull qwen3:4b-q4_K_M # explicit Q4_K_M
ollama pull llama3.2:3b-q5_K_M # explicit Q5_K_M
```

The default tag (e.g. qwen3:4b) pulls Q4_K_M automatically for most models. This is the correct starting point for all 4GB GPU users.

Verifying a Model's VRAM Usage

After pulling and running a model, check actual VRAM usage with:

```
nvidia-smi
```

Look at the 'MEM-Usage' column for the ollama process. If VRAM usage equals or exceeds your total VRAM, the model is too large and is falling back to CPU.

Phase 1: Making an agent

Finding All Available Tags for a Model

To see all available quantization variants for a model, visit the Ollama library:

```
https://ollama.com/library/<model_name>/tags
```

For example:

```
https://ollama.com/library/qwen3/tags
https://ollama.com/library/llama3.2/tags
```

Note: *Not all quantization variants are available for every model. If a specific variant is not listed, use the default tag which Ollama maps to the best available quantization for your hardware.*

Listing and Managing Installed Models

Task	Command
List all installed models	ollama list
Remove a model to free disk space	ollama rm <model_tag>
Check model details	ollama show <model_tag>
Run a model interactively	ollama run <model_tag>
Check VRAM usage during inference	nvidia-smi

Phase 1: Making an agent

Diagnostics Command Reference

These commands are referenced repeatedly throughout this guide. Familiarize yourself with them before starting.

What to Check	Command	What to Look For
RAM usage	<code>free -h</code>	Available memory under the 'available' column. You want at least 4–6 GB free before loading a model.
CPU usage (live)	<code>Top</code>	Press Q to exit. Check %CPU and %MEM columns for active processes.
CPU usage (better)	<code>Htop</code>	If installed. Color-coded and easier to read than top.
GPU status + VRAM	<code>nvidia-smi</code>	Shows GPU utilization % and memory used/total. Expect 0 MB used when no model is loaded.
Process memory list	<code>ps aux --sort=-%mem head -20</code>	Lists top 20 processes by memory consumption.
Disk usage	<code>df -h</code>	Check available space on the partition where models are stored (~/.ollama/).
Ollama process check	<code>ps aux grep ollama</code>	Confirms Ollama is running as a background process.

Prerequisites

Before running any install commands, confirm the following are already true. These are not installed in this guide — they are assumed to exist based on your hardware and environment setup.

Prerequisite	How to Verify	Expected Result
WSL2 is installed and running Debian	Run in PowerShell: <code>wsl --list --verbose</code>	Debian listed with VERSION = 2 and STATE = Running

Phase 1: Making an agent

Prerequisite	How to Verify	Expected Result
CUDA libraries are present in WSL2	Run in WSL2: <code>nvcc --version</code>	Shows CUDA compiler version — any version is acceptable
NVIDIA GPU is visible in WSL2	Run in WSL2: <code>nvidia-smi</code>	Shows RTX 4050 with 6144 MiB total memory
Python 3.12.3 is active	Run in WSL2: <code>python3 --version</code>	Output: Python 3.12.3
pip is available	Run in WSL2: <code>pip --version</code>	Shows pip version linked to Python 3.12
Internet connection is active	Run in WSL2: <code>curl -I https://ollama.com</code>	Returns HTTP 200 or 301 — any response means connectivity is working
At least 20 GB free disk space	Run in WSL2: <code>df -h ~</code>	Available column shows 20G or more (models are large)

Warning: *If `nvidia-smi` fails inside WSL2, CUDA passthrough is not working. Do not proceed — the models will run on CPU only, which will be extremely slow on your hardware. Fix this first.*

Note: *In my case, `pip` refuses to run, so I use `pip3`.*

WSL is not required but is recommended. If your python environment has access to CUDA and your GPU you may proceed.

Phase 1: Making an agent

Install Ollama

The local model runtime that manages and serves your LLMs via a local API.

Action

```
curl -fsSL https://ollama.com/install.sh | sh
```

This downloads and runs the official Ollama install script. It will install the Ollama binary, create a system service, and start Ollama automatically.

Test

After the install completes, run:

```
ollama --version
```

Expected output: a version string such as

```
ollama version 0.x.x
```

```
lenidar@Lenidar:/mnt/c/Users/Lenidar$ ollama --version
ollama version is 0.20.7
```

Then confirm the Ollama service is running:

```
systemctl status ollama
```

Expected: status shows **active (running)**. If systemctl is not available in your WSL2 setup, run:

```
ps aux | grep ollama
```

Expected: an ollama process appears in the list.

```
lenidar@Lenidar:/mnt/c/Users/Lenidar$ systemctl status ollama
● ollama.service - Ollama Service
   Loaded: loaded (/etc/systemd/system/ollama.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-04-17 08:13:48 PST; 55s ago
     Main PID: 1935 (ollama)
        Tasks: 11 (limit: 11931)
       Memory: 115.6M ()
      CGroup: /system.slice/ollama.service
              └─1935 /usr/local/bin/ollama serve
```

Diagnostics

```
free -h
```

Ollama running as a background service uses very little RAM on its own (under 100 MB). Your available RAM should not have dropped significantly.

```
nvidia-smi
```

No model is loaded yet. GPU memory used should show 0 MiB or close to it.

Phase 1: Making an agent

Note: If the Ollama service did not start automatically, start it manually with: `ollama serve` — run this in a separate terminal and leave it open.

```
lenidar@lenidar: /mnt/c/Users/Lenidar$ nvidia-smi
Fri Apr 17 08:15:55 2026
```

NVIDIA-SMI 595.58.02		Driver Version: 595.97		CUDA Version: 13.2	
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile
Fan	Temp	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Uncorr. Compute M.
Perf					MIG M.
0	NVIDIA GeForce RTX 4050	On	00000000:01:00.0	Off	N/A
N/A	39C	3W / 140W	125MiB / 6141MiB	0%	Default
P8					N/A

Processes:						
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage
ID	ID					
No running processes found						

Phase 1: Making an agent

Pull Llama 3.1 8B

The orchestrator model — the primary brain of your agent.

Action

```
ollama pull llama3.1:8b
```

Note: Or you know... Whatever your orchestrator model of choice is.

This downloads the Llama 3.3 8B model at default quantization (Q4_K_M). The download is approximately 5 GB. Download time depends on your internet connection.

```
lenidar@Lenidar:/mnt/c/Users/Lenidar$ ollama pull llama3.1:8b
pulling manifest
pulling 667b0c1932bc: 7% ██████████ | 348 MB/4.9 GB 749 KB/s 1h41m
```

Warning: Do not close the terminal during the download. If it is interrupted, re-run the same command — Ollama resumes partial downloads.

Test

After the download completes, verify the model is registered:

```
ollama list
```

Expected: llama3.1:8b appears in the list with its size and modification date.

```
lenidar@Lenidar:/mnt/c/Users/Lenidar$ ollama list
NAME                ID                SIZE      MODIFIED
llama3.1:8b         46e0c10c039e     4.9 GB    32 seconds ago
```

Then run a quick inference test to confirm the model loads and responds:

```
ollama run llama3.1:8b "Reply with the single word: READY"
```

Expected output: READY (or a very short response). This confirms the model loads into VRAM and generates output correctly.

```
lenidar@Lenidar:/mnt/c/Users/Lenidar$ ollama run llama3.1:8b "Reply with the single word: Bazinga!"
Bazinga!
```

After the test, exit the model session:

```
/bye
```

Slight modification to the code

There is a slight problem that does not unload the model from VRAM after use, so we are going to do a modification of its service settings.

Run this code to open the services file

```
sudo nano /etc/systemd/system/ollama.service
```

Add the following line to the text editor that opens

```
Environment="OLLAMA_KEEP_ALIVE=0"
```

Phase 1: Making an agent

```
[Service]
ExecStart=/usr/local/bin/ollama serve
User=ollama
Group=ollama
Restart=always
RestartSec=3
Environment="PATH=/home/lenidar/.local/bin:/usr/local/bin:/usr/bin:/usr/sbin:/bin:/sbin"
Environment="OLLAMA_KEEP_ALIVE=0"

[Install]
WantedBy=default.target
```

Restart the model

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

Verify that the model is still running correctly

```
sudo systemctl status ollama
```

Diagnostics

Run these immediately after the model exits (before VRAM is fully released):

```
nvidia-smi
```

While the model is loaded you should see approximately 5000–5500 MiB of your 6144 MiB VRAM in use. After /bye, usage should return close to 0 MiB within a few seconds.

```
free -h
```

RAM usage should return close to its pre-load baseline after the model exits. If RAM does not recover, Ollama may be keeping the model partially resident — this is normal behaviour and not a problem.

```
df -h ~
```

Confirm the model files are stored and disk space has decreased by approximately 5 GB.



Note: This is what I used for diagnostics, I ran task manager before running the script then just observed. The spike is the loading and unloading of the model which verifies the changes.

Here is a second test just to test if it still works

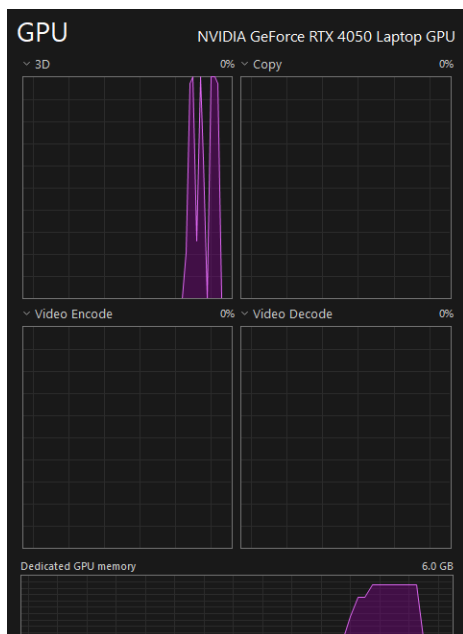
Phase 1: Making an agent

```
lenidar@Lenidar:/mnt/c/Users/Lenidar$ ollama run llama3.1:8b "May I know what the answer of 1+1 is?"
The answer to 1 + 1 is 2.

lenidar@Lenidar:/mnt/c/Users/Lenidar$ |
```

A screenshot of a GPU monitoring interface showing a sharp spike in activity. The interface includes a grid for various metrics and a 'Dedicated GPU memory' section with a 6.0 GB total capacity. The spike indicates a moment of high GPU utilization.

Testing for hallucination and hardware usage:



This is using a json set of instructions to generate and concatenate text into a cohesive paragraph. This short spike is the model thinking and displaying the output.

```
cat prompt.json | ollama run llama3.1:8b --verbose
```

```
total duration:      17.179254173s
load duration:       9.977495742s
prompt eval count:    347 token(s)
prompt eval duration: 317.934674ms
prompt eval rate:     1091.42 tokens/s
eval count:           143 token(s)
eval duration:        6.781569494s
eval rate:            21.09 tokens/s
```

The ‘—verbose’ shows the metrics of the model AFTER it runs, here is a sample response. Please remember that we are running locally, the screenshot you are about to see actually surprised me it reached 21 TPS.

Phase 1: Making an agent

Pull Qwen2.5-Coder 7B (Optional: If you need a second model)

Note: Or whatever other model you need. This is a preparation step, you may opt to skip this for now if you so wish to do so.

The Python library that lets your orchestration scripts talk to Ollama's local API.

Action

```
pip3 install ollama --break-system-packages
```

This installs the official Ollama Python client. The `--break-system-packages` flag is required on Debian 12+ with Python 3.12 due to PEP 668 environment protections.

Note: No screenshot too much text to say it worked. I'll just show the success message

```
Successfully installed annotated-types-0.7.0 anyio-4.13.0 h11-0.16.0 httpcore-1.0.9 httpx-0.28.1 ollama-0.6.1 pydantic-2.13.2 pydantic-core-2.46.2 typing-extensions-4.15.0 typing-inspection-0.4.2
lenidar@lenidar:/mnt/c/Users/Lenidar$
```

Test

Confirm the package installed correctly:

```
pip3 show ollama
```

Expected: pip identifies ollama as a valid library and outputs its details.

```
lenidar@lenidar:/mnt/c/Users/Lenidar$ pip3 show ollama
Name: ollama
Version: 0.6.1
Summary: The official Python client for Ollama.
Home-page:
Author:
Author-email: hello@ollama.com
License:
Location: /home/lenidar/.local/lib/python3.12/site-packages
Requires: httpx, pydantic
Required-by:
```

Then run a functional test — this sends a real API call to your local Ollama instance:

```
python3 -c "import ollama; r = ollama.chat(model='llama3.1:8b',
messages=[{'role':'user','content':'Say READY'}]);
print(r['message']['content'])"
```

Expected: the model responds with READY or a short acknowledgement. This confirms Python can reach Ollama's local API and get a response.

```
lenidar@lenidar:/mnt/c/Users/Lenidar$ python3 -c "import ollama; r = ollama.chat(model='llama3.1:8b',
messages=[{'role':'user','content':'Say READY'}]); print(r['message']['content'])"
READY!
```

Note: Running your models for the first time results in weird responses, just run the script again if it does happen to verify. They call it a COLD-START bug. Here is my first attempt at running the code...

```
lenidar@lenidar:/mnt/c/Users/Lenidar$ python3 -c "import ollama; r = ollama.chat(model='llama3.1:8b',
messages=[{'role':'user','content':'Say READY'}]); print(r['message']['content'])"
SET...
```

Phase 1: Making an agent

Diagnostics

```
nvidia-smi
```

During the Python test call above, the model will load into VRAM briefly. If you run nvidia-smi fast enough, you will see VRAM spike. After the call completes, it should settle back down.

```
free -h
```

RAM usage should be stable. The Python process itself is lightweight.

```
pip show ollama
```

Confirms the package is installed, shows version and location.

Note: *If you did the slight modification change which unloads the model, you should not need the diagnostics because it still unloads the model from VRAM*

Phase 1: Making an agent

Create the Project Folder Structure

Set up the directories and placeholder files the agent will use.

Action

Run the following commands in order. Each creates a directory or file in your home directory:

```
mkdir -p ~/local_agent
mkdir -p ~/agent_memory
```

```
lenidar@Lenidar:~$ ls
Python-3.12.9  Python-3.12.9.tgz  PythonScripts  agent_memory  local_agent  prompts  snap  tattoo
```

Create the four memory files:

```
touch ~/agent_memory/user_profile.md
touch ~/agent_memory/playbook.md
touch ~/agent_memory/session_notes.md
touch ~/agent_memory/projects.md
```

```
lenidar@Lenidar:~$ cd agent_memory
lenidar@Lenidar:~/agent_memory$ ls
playbook.md  projects.md  session_notes.md  user_profile.md
```

Create the Python script placeholders:

```
touch ~/local_agent/main.py
touch ~/local_agent/memory.py
touch ~/local_agent/ollama_client.py
touch ~/local_agent/config.py
```

```
lenidar@Lenidar:~/agent_memory$ touch ~/local_agent/main.py
touch ~/local_agent/memory.py
touch ~/local_agent/ollama_client.py
touch ~/local_agent/config.py
lenidar@Lenidar:~/agent_memory$ cd ../local_agent
lenidar@Lenidar:~/local_agent$ ls
config.py  main.py  memory.py  ollama_client.py
```

Test

Note: You CAN do this, but I didn't have to as you seen in my screenshots I just navigated to each location and listed them out.

Verify the folder structure exists and all files are present:

```
find ~/local_agent ~/agent_memory -type f | sort
```

Expected output — all eight files listed:

```
/home/<your_username>/agent_memory/playbook.md
/home/<your_username>/agent_memory/projects.md
/home/<your_username>/agent_memory/session_notes.md
/home/<your_username>/agent_memory/user_profile.md
```

Phase 1: Making an agent

```
/home/<your_username>/local_agent/config.py  
/home/<your_username>/local_agent/main.py  
/home/<your_username>/local_agent/memory.py  
/home/<your_username>/local_agent/ollama_client.py
```

Diagnostics

```
df -h ~
```

These are empty files — disk usage should not have changed meaningfully. If it has, something else is writing to your home directory.

Note: *The memory files are intentionally empty at this stage. You will populate `user_profile.md` and `playbook.md` in the next step.*

Phase 1: Making an agent

Populate the Memory Files

Write Level 1 (user profile) and Level 2 (playbook) memory content.

Action — Level 1: user_profile.md

Open the file in any text editor available in WSL2:

```
nano ~/agent_memory/user_profile.md
```

Paste the following and fill in your details:

```
# User Profile

Name: Julian
Role: Instructor and Developer

## Teaching Subjects
- C# (Console Applications)
- C# (WPF — Windows Presentation Foundation)
- Python (scripting and small AI projects)
- Currently learning: MAUI (personal, not taught yet)

## Working Style
- Does not expect full solutions from the agent
- Needs: scaffolding, boilerplate, debugging help, module assessments
- Uses the agent as a local teaching assistant
- May ask for prompts to use with online AI models for document tasks

## Environment
- WSL2 Debian, Python 3.12.3
- .NET development on Windows side
- RTX 4050 laptop GPU
```

Save and exit nano: press Ctrl+O, Enter, then Ctrl+X.

Note: This is how I populated it, you may choose to change it, I may change it too as the project goes on but this is my initial user profile.

THIS TELLS YOUR AGENT WHO YOU ARE AND WHAT YOU NEED FROM IT.

Action — Level 2: playbook.md

```
nano ~/agent_memory/playbook.md
```

Paste the following exactly:

```
# Agent Playbook — Prime Directive

## Identity
```

Phase 1: Making an agent

```
You are a local teaching assistant agent for Julian.
You are not a code generator. You are a thinking partner and task router.

## Core Rules
1. Always read the user profile before responding.
2. Ask clarifying questions before delegating or producing output.
3. Rewrite vague prompts into structured, specific requests.
4. Never produce complete solutions – scaffolding and guidance only.
5. Keep responses concise and technical. Avoid unnecessary filler.

## Task Routing
- Coding help (C#, WPF, MAUI, Python) → flag for coding sub-agent (Phase 2)
- Prompt generation for online models → handle directly
- General teaching assistance → handle directly
- Unclear intent → ask one clarifying question before doing anything

## Response Style
- Plain text only. No markdown formatting in terminal output.
- Be direct. Do not repeat the user's prompt back at them.
- If you need more information, ask exactly one question at a time.
```

Note: *This is what makes your agent the agent. Honestly, I spent 1 week on this.*

Save and exit: Ctrl+O, Enter, Ctrl+X.

Test

Verify both files have content:

```
wc -l ~/agent_memory/user_profile.md ~/agent_memory/playbook.md
```

Expected: both files show a line count greater than 0.

Read them back to confirm content is intact:

```
cat ~/agent_memory/user_profile.md
cat ~/agent_memory/playbook.md
```

Diagnostics

```
ls -lh ~/agent_memory/
```

All four files should be listed. user_profile.md and playbook.md should show a non-zero file size. session_notes.md and projects.md should show 0 bytes — that is correct for now.

Phase 1: Making an agent

Write the Core Python Scripts

The four Python files that form the agent's runtime.

Action — config.py

Open the file:

```
nano ~/local_agent/config.py
```

Paste the following:

```
import os

# Model configuration
ORCHESTRATOR_MODEL = "llama3.1:8b"

# Memory file paths
MEMORY_DIR = os.path.expanduser("~/agent_memory")
USER_PROFILE_PATH = os.path.join(MEMORY_DIR, "user_profile.md")
PLAYBOOK_PATH = os.path.join(MEMORY_DIR, "playbook.md")
SESSION_NOTES_PATH = os.path.join(MEMORY_DIR, "session_notes.md")
PROJECTS_PATH = os.path.join(MEMORY_DIR, "projects.md")

# Ollama API
OLLAMA_HOST = "http://localhost:11434"
```

Save and exit.

Note: Port number 11434 is the default Ollama server port

Action — memory.py

```
nano ~/local_agent/memory.py
```

Paste the following:

```
from config import USER_PROFILE_PATH, PLAYBOOK_PATH

def load_memory():
    """Load Level 1 and Level 2 memory files and return as a single
    string."""
    sections = []

    for label, path in [("USER PROFILE", USER_PROFILE_PATH),
                        ("PLAYBOOK", PLAYBOOK_PATH)]:
        try:
            with open(path, "r", encoding="utf-8") as f:
                content = f.read().strip()
```

Phase 1: Making an agent

```
        sections.append(f"--- {label} ---\n{content}")
    except FileNotFoundError:
        sections.append(f"--- {label} ---\n[FILE NOT FOUND: {path}]")

    return "\n\n".join(sections)
```

Save and exit.

Action — ollama_client.py

```
nano ~/local_agent/ollama_client.py
```

Paste the following:

```
import ollama
from config import ORCHESTRATOR_MODEL

def chat(messages: list) -> str:
    """Send message history to the orchestrator model and return the
    response text."""
    response = ollama.chat(
        model=ORCHESTRATOR_MODEL,
        messages=messages
    )
    return response["message"]["content"]
```

Save and exit.

Action — main.py

```
nano ~/local_agent/main.py
```

Paste the following:

```
import sys
import os
sys.path.insert(0, os.path.dirname(__file__))

from memory import load_memory
from ollama_client import chat

def build_system_prompt(memory: str) -> str:
    return (
        "You are a local teaching assistant agent.\n\n"
        + memory
    )

def main():
```


Phase 1: Making an agent

```
print("Loading memory...")
memory = load_memory()
system_prompt = build_system_prompt(memory)

messages = [{"role": "system", "content": system_prompt}]

print("Agent ready. Type your message. Press Ctrl+C to exit.\n")

while True:
    try:
        user_input = input("You: ").strip()
    except (KeyboardInterrupt, EOFError):
        print("\nSession ended.")
        break

    if not user_input:
        continue

    messages.append({"role": "user", "content": user_input})

    print("Agent: ", end="", flush=True)
    response = chat(messages)
    print(response)
    print()

    messages.append({"role": "assistant", "content": response})

if __name__ == "__main__":
    main()
```

Save and exit.

Test

Verify all four files have content:

```
wc -l ~/local_agent/*.py
```

All four files should show a line count greater than 0. None should be 0 lines.

```
lenidar@lenidar:~/local_agent$ wc -l ~/local_agent/*.py
 14 /home/lenidar/local_agent/config.py
 43 /home/lenidar/local_agent/main.py
 16 /home/lenidar/local_agent/memory.py
 10 /home/lenidar/local_agent/ollama_client.py
 83 total
```

Phase 1: Making an agent

Diagnostics

```
ls -lh ~/local_agent/
```

All four .py files should show non-zero file sizes.

```
lenidar@Lenidar:~/local_agent$ ls -lh ~/local_agent/
total 16K
-rw-r--r-- 1 lenidar lenidar 442 Apr 18 12:45 config.py
-rw-r--r-- 1 lenidar lenidar 1.1K Apr 18 12:48 main.py
-rw-r--r-- 1 lenidar lenidar 603 Apr 18 12:46 memory.py
-rw-r--r-- 1 lenidar lenidar 309 Apr 18 12:47 ollama_client.py
```

Phase 1: Making an agent

Run the Agent — First Launch

Start the agent and confirm it loads memory and responds correctly.

Action

```
cd ~/local_agent && python3 main.py
```

Test — T1: Memory Loads

The terminal should print:

```
Loading memory...
Agent ready. Type your message. Press Ctrl+C to exit.
```

At the prompt, type:

```
What do you know about me?
```

Pass condition: the agent responds with information from `user_profile.md` — your name, teaching subjects, and working style — without you having typed any of it in this session.

Fail condition: the agent says it has no information, gives a generic response, or throws an error.

```
lenidar@lenidar:~/local_agent$ cd ~/local_agent && python3 main.py
Loading memory...
Agent ready. Type your message. Press Ctrl+C to exit.

You: What do you know about me?
Agent: You are [REDACTED], an Instructor and Developer. I have access to your User Profile, which tells me that you teach and develop in various programming languages, including C#, Python, and currently learning MAUI. [REDACTED]

You work with a Debian environment (WSL2) for Python development and use Windows for .NET development. Your laptop is equipped with an RTX 4050 GPU.

You expect me to provide scaffolding, boilerplate, debugging help, and module assessments as a local teaching assistant, rather than full solutions. You may also ask me to generate prompts for online AI models for document tasks.

How can I assist you today?

You:
```

Note: Redacted info, things I modified HEHE.

Test — T2: Playbook Governs Behaviour

Type a vague prompt:

```
Help me with something in C#.
```

Pass condition: the agent asks a clarifying question before offering any help.

Fail condition: the agent immediately produces code, advice, or a generic response without asking what you need.

```
You: Help me with something in c#
Agent: I'll flag this task for the coding sub-agent. Before they help you, what specifically do you need assistance with in C# (e.g., Console Applications, WPF, MAUI)? Are you working on a particular project or facing an issue?
```

Phase 1: Making an agent

Note: *This is so cool NGL...*

Test — T3: Multi-Turn Memory

Tell the agent something new:

```
I am currently working on a WPF project called ClassTracker.
```

```
You: im currently woking on a WPF project called class tracker, it has an HTML front end too.  
Agent: So you're building a Class Tracker application with a WPF backend and an HTML frontend. That's interesting.
```

```
Can you tell me what specific aspect of the C# codebase in your WPF project is giving you trouble? Is it related to data binding, event handling, or something else entirely?
```

```
Additionally, since you mentioned an HTML frontend, I'll assume that there might be some interaction between the two parts. Are you using any libraries or frameworks (e.g., Blazor WebAssembly) for this integration, or just plain old JavaScript and AJAX calls?
```

Then later in the same session, ask:

```
What project am I working on?
```

Pass condition: the agent correctly recalls ClassTracker from earlier in the session.

Fail condition: the agent says it does not know or gives a wrong answer.

```
You: what project am i working on  
Agent: You're currently working on a Class Tracker application, which has a WPF backend and an HTML frontend. It's a personal project you're developing as part of your teaching and development work.
```

Test — T4: Prompt Generation

Ask:

```
Generate a prompt I can use with an online model to help me write a lesson plan for introducing WPF data binding to beginners.
```

Pass condition: the agent produces a detailed, context-aware prompt that references your teaching level, subject, and audience — not a generic template.

Fail condition: the agent produces a vague, generic prompt that ignores your context.

```
You: Generate a prompt I can use with an online model to help me write a lesson plan for introducing WPF data binding to beginners.
```

```
Agent: Here's a prompt you can use:
```

```
"Create a lesson plan outline for teaching WPF data binding basics to beginner students, including:
```

- * Explanation of what data binding is and its importance in WPF
- * Step-by-step guide on setting up a simple data-bound UI component (e.g., TextBlock)
- * Introduction to XAML syntax and how it relates to data binding
- * Hands-on exercise or example project for students to practice implementing data binding

```
Assume the students have basic knowledge of C# and WPF fundamentals. Keep the language accessible and concise, as if explaining it to a group of beginners.
```

```
Format the response as an outline with clear headings and bullet points."
```

```
You can copy-paste this prompt into your online AI model's input field and see what lesson plan ideas come out!
```

Phase 1: Making an agent

Note: *This response is good because it knows NOT to generate it but ask for help from a different model. I have to tweak its prompt generation but good nevertheless.*

Diagnostics — During Active Session

Open a second WSL2 terminal while the agent is running and execute:

```
nvidia-smi
```

Expected: approximately 5000–5500 MiB of VRAM in use while the model is loaded and responding.

```
free -h
```

RAM usage will be elevated while the model is active. This is normal.

```
ps aux --sort=-%mem | head -10
```

The ollama process should appear near the top of the list.

Diagnostics — After Session Ends (Ctrl+C)

```
nvidia-smi
```

VRAM usage should return close to 0 MiB within a few seconds of the session ending.

```
free -h
```

RAM should recover toward its pre-session baseline.

Note: *I suggest running it against a trusted LLM, for example I used copilot and just asked it the same things then compared. I also had copilot asses the answers of my agent.*